

# Arbitrary Precision Arithmetic Library in Java

Design, Implementation, Testing, and Deployment of  
Arbitrary-Precision Integer and Floating-Point Arithmetic

Atul Boyal  
Indian Institute of Technology Hyderabad

## Abstract

Modern programming languages provide fixed-width numeric data types that suffer from overflow and precision limitations when handling extremely large numbers. This project presents the design and implementation of an Arbitrary Precision Arithmetic Library in Java capable of performing addition, subtraction, multiplication, and division on integers and floating-point numbers of virtually unlimited size.

The library consists of three major components: **AInteger**, **AFloat**, and **MyInfArith**. Integer arithmetic is implemented using manual digit-by-digit algorithms operating on normalized string representations, while floating-point arithmetic is built on top of the integer engine through decimal alignment and scaling techniques.

The project includes input validation, sign management, normalization, a command-line interface, Maven-based build automation, Docker containerization, and a comprehensive JUnit test suite containing 68 passing test cases. The resulting system demonstrates the feasibility of implementing arbitrary-precision arithmetic without relying on Java's built-in **BigInteger** or **BigDecimal** classes.

# Contents

---

|           |                                           |          |
|-----------|-------------------------------------------|----------|
| <b>1</b>  | <b>Introduction</b>                       | <b>3</b> |
| 1.1       | Motivation . . . . .                      | 3        |
| 1.2       | Objectives . . . . .                      | 3        |
| <b>2</b>  | <b>System Architecture</b>                | <b>3</b> |
| 2.1       | High-Level Design . . . . .               | 3        |
| 2.2       | Class Responsibilities . . . . .          | 4        |
| 2.2.1     | AInteger . . . . .                        | 4        |
| 2.2.2     | AFloat . . . . .                          | 4        |
| 2.2.3     | MyInfArith . . . . .                      | 4        |
| <b>3</b>  | <b>AInteger Design and Implementation</b> | <b>4</b> |
| 3.1       | Internal Representation . . . . .         | 4        |
| 3.2       | Input Validation . . . . .                | 4        |
| 3.3       | Normalization . . . . .                   | 5        |
| 3.4       | Sign Handling . . . . .                   | 5        |
| <b>4</b>  | <b>Arithmetic Algorithms</b>              | <b>5</b> |
| 4.1       | Addition . . . . .                        | 5        |
| 4.2       | Subtraction . . . . .                     | 6        |
| 4.3       | Multiplication . . . . .                  | 6        |
| 4.4       | Division . . . . .                        | 6        |
| <b>5</b>  | <b>AFloat Design and Implementation</b>   | <b>6</b> |
| 5.1       | Core Design Principle . . . . .           | 6        |
| 5.2       | Decimal Alignment . . . . .               | 7        |
| 5.3       | Scaling Technique . . . . .               | 7        |
| <b>6</b>  | <b>Complexity Analysis</b>                | <b>7</b> |
| <b>7</b>  | <b>Testing and Verification</b>           | <b>7</b> |
| 7.1       | Coverage . . . . .                        | 7        |
| 7.2       | Test Summary . . . . .                    | 8        |
| <b>8</b>  | <b>Build and Deployment</b>               | <b>8</b> |
| 8.1       | Maven . . . . .                           | 8        |
| 8.2       | Docker . . . . .                          | 8        |
| <b>9</b>  | <b>Results</b>                            | <b>8</b> |
| <b>10</b> | <b>Engineering Challenges</b>             | <b>9</b> |
| <b>11</b> | <b>Future Work</b>                        | <b>9</b> |
| <b>12</b> | <b>Conclusion</b>                         | <b>9</b> |

# 1 Introduction

---

## 1.1 Motivation

Programming languages typically represent numbers using fixed-width primitive types such as `int`, `long`, `float`, and `double`. While efficient, these representations suffer from two major limitations:

1. Integer overflow when values exceed storage capacity.
2. Precision loss in floating-point computations.

Applications involving scientific computing, cryptography, finance, and numerical analysis often require arithmetic operations on values far larger than primitive types can represent. This motivates the need for arbitrary-precision arithmetic.

## 1.2 Objectives

The primary objectives of this project are:

- Implement arbitrary-precision integer arithmetic.
- Implement arbitrary-precision floating-point arithmetic.
- Support addition, subtraction, multiplication, and division.
- Handle positive and negative numbers.
- Validate and normalize user input.
- Provide a command-line interface.
- Ensure correctness through automated testing.

# 2 System Architecture

---

## 2.1 High-Level Design

The system consists of three principal classes:

- `AInteger`
- `AFloat`
- `MyInfArith`

The architecture consists of three primary classes:

- `MyInfArith` – Command-line interface.
- `AInteger` – Arbitrary precision integer arithmetic.
- `AFloat` – Arbitrary precision floating-point arithmetic.

## 2.2 Class Responsibilities

### 2.2.1 AInteger

Responsible for arbitrary-precision integer arithmetic.

Features include:

- Input validation
- Normalization
- Sign handling
- Addition
- Subtraction
- Multiplication
- Division

### 2.2.2 AFloat

Responsible for arbitrary-precision floating-point arithmetic.

Features include:

- Decimal alignment
- Scaling to integer representations
- Reuse of integer arithmetic algorithms

### 2.2.3 MyInfArith

Provides a command-line interface for interacting with the library.

## 3 AInteger Design and Implementation

---

### 3.1 Internal Representation

Each integer is represented using a normalized string:

```
private String value;
```

This approach eliminates limitations imposed by primitive numeric types.

### 3.2 Input Validation

Validation is performed using regular expressions.

Accepted examples:

- 12345
- -67890

- 0

Rejected examples:

- abc
- 12.34
- - - 45

### 3.3 Normalization

Normalization ensures canonical representation.

Examples:

| Input  | Normalized Output |
|--------|-------------------|
| 000123 | 123               |
| -00045 | -45               |
| 0000   | 0                 |
| -0000  | 0                 |

Table 1: Normalization Examples

### 3.4 Sign Handling

All arithmetic operations reduce to combinations of positive arithmetic and sign management.

Cases considered include:

- (+,+)
- (+,-)
- (-,+)
- (-,-)

## 4 Arithmetic Algorithms

---

### 4.1 Addition

Addition follows the traditional carry-propagation algorithm.

1. Traverse digits from right to left.
2. Add corresponding digits and carry.
3. Store resulting digit.
4. Propagate carry.

Time Complexity:

$$O(n)$$

## 4.2 Subtraction

Subtraction follows the traditional borrow-propagation algorithm.

1. Compare operands.
2. Borrow when required.
3. Compute digit differences.

Time Complexity:

$$O(n)$$

## 4.3 Multiplication

Multiplication uses the classical long multiplication algorithm.

1. Generate partial products.
2. Apply positional shifts.
3. Accumulate results.

Time Complexity:

$$O(n^2)$$

## 4.4 Division

Division uses long division.

1. Bring down digits.
2. Determine quotient digit.
3. Subtract divisor multiple.
4. Repeat until completion.

Time Complexity:

$$O(n^2)$$

# 5 AFloat Design and Implementation

---

## 5.1 Core Design Principle

Rather than implementing separate arithmetic algorithms, floating-point arithmetic reuses the integer arithmetic engine.

## 5.2 Decimal Alignment

Example:

$$12.34 + 5.6$$

becomes

$$1234 + 560$$

after decimal alignment.

## 5.3 Scaling Technique

The workflow is:

1. Remove decimal points.
2. Align decimal places.
3. Perform integer arithmetic.
4. Restore decimal point.
5. Normalize result.

# 6 Complexity Analysis

---

| Operation      | Complexity |
|----------------|------------|
| Addition       | $O(n)$     |
| Subtraction    | $O(n)$     |
| Multiplication | $O(n^2)$   |
| Division       | $O(n^2)$   |

Table 2: Complexity Analysis

# 7 Testing and Verification

---

The project includes comprehensive automated testing using JUnit 5.

## 7.1 Coverage

Testing covers:

- Constructors
- Parsing
- Validation

- Normalization
- Large numbers
- Negative values
- Floating-point arithmetic
- Division by zero
- Edge cases

## 7.2 Test Summary

| Metric      | Value |
|-------------|-------|
| Total Tests | 68    |
| Failures    | 0     |
| Errors      | 0     |

Table 3: Test Summary

## 8 Build and Deployment

---

### 8.1 Maven

Build commands:

```
mvn clean test
mvn clean package
```

### 8.2 Docker

Build:

```
docker build -t arbitrary-precision-arithmetic .
```

Run:

```
docker run arbitrary-precision-arithmetic int add 123 456
```

Output:

```
579
```

## 9 Results

---

Example integer addition:

```
23650078224912949497310933240250
+
42939783262467113798386384401498
```



=  
66589861487380063295697317641748

Example floating-point division:

$5.5 / 2 = 2.75$

The implementation successfully handles values significantly larger than Java primitive numeric types.

## 10 Engineering Challenges

---

Several engineering challenges were encountered during development:

1. Sign handling across arithmetic operations.
2. Preventing invalid representations such as -0.
3. Decimal alignment in floating-point arithmetic.
4. Long division implementation.
5. Maintaining correctness through extensive testing.

## 11 Future Work

---

Potential enhancements include:

- Karatsuba multiplication
- Faster division algorithms
- Modulo operation
- Exponentiation
- Scientific notation support
- User-configurable precision

## 12 Conclusion

---

This project successfully implements arbitrary-precision integer and floating-point arithmetic in Java without relying on built-in arbitrary-precision libraries. Through string-based representations, manual arithmetic algorithms, automated testing, Maven packaging, and Docker deployment, the resulting system provides a complete and extensible foundation for high-precision numerical computation.